

D1. Selection rationale: câu "Why"

Tình huống kết quả	Câu mẫu (English)
Ensemble/boosting hơn hẳn Logistic	[Model] achieved the highest validation F1 for label 1 ([x] vs [y] for logistic regression) and ROC_AUC ([a] vs [b]), suggesting that non-linear effects and feature interactions matter in this problem and cannot be captured by a linear model.
Hai model sát nhau	[A] and [B] differ by only [x] in F1, which is within the fold-to-fold standard deviation ([sd]). We selected [A] because it trains [n] times faster / is less sensitive to hyperparameters / shows a smaller train-validation gap.
Logistic gần bằng model tốt nhất	Logistic regression came within [x] F1 of the best ensemble and is fully interpretable, but we selected [model] because the gain in F1 for the bad-debt class outweighed the loss of transparency. (Nếu chọn Logistic: "...we selected logistic regression because the difference was within noise and it is simpler to explain and audit.")
ROC_AUC và F1 chọn khác model	Model [A] had the higher ROC_AUC ([x] vs [y]), but [B] achieved better F1 for label 1 ([c] vs [d]). Because bad debt is the minority class, ROC_AUC can look high due to the many true negatives, so we gave more weight to F1 and precision on the positive class.
Class weight tăng F1 nhưng ROC_AUC gần như không đổi	Class weighting raised F1 for label 1 from [a] to [b] but left ROC_AUC largely unchanged ([c] vs [d]); reweighting mainly shifts predicted probabilities and hence the operating point rather than how well customers are ranked, so we handled the precision-recall balance through the threshold.
Train rất cao, Valid thấp (overfit)	[Model] fits the training data almost perfectly (train F1 [x], ROC_AUC [y]) but validation F1 is [z]; this gap indicates overfitting. Constraining [max_depth / min_samples_leaf] reduced the gap to [g] while improving validation F1.
Tuning chỉ cải thiện rất ít	Hyperparameter tuning improved validation F1 by only [x], within cross-validation noise, so we [kept the tuned model for its smaller train-validation gap / retained default settings for simplicity].
F1 tăng do recall tăng, precision giảm	The higher F1 of [A] comes mainly from recall ([r1] vs [r2]) at the cost of precision ([p1] vs [p2]); this fits a lending setting where missed bad debt is costlier than a false alarm.
Giải thích baseline	The dummy baseline has ROC_AUC of about 0.5 and F1 of [x] for label 1, so every model above it extracts real signal from the features.
Chọn model chậm hơn nhưng tốt hơn	[Model] is [n] times slower to train than [runner-up] ([s] s vs [s] s), which is acceptable at this data size given its [x] higher validation F1.

D2. False negatives và false positives (bối cảnh cho vay)

Mục	Câu mẫu (English)
False negative (đầy đủ)	Missed bad-debt customers ([fn] of [P] in validation, recall [r] at threshold [t]) would be approved for credit they cannot repay, causing principal losses, higher provisioning and collection costs, and deterioration of the loan portfolio.
False negative + ROC_AUC	ROC_AUC ([x]) shows that the model ranks bad debts above good customers well overall, but at this cutoff [fn] bad-debt cases still slip through.

False negative (ngắn)	Each missed bad debt is a direct credit loss, which is usually larger than the margin lost on a wrongly rejected good customer.
False positive (đầy đủ)	False alarms ([fp] customers, precision [p]) would lead to good customers being rejected, given lower limits, or sent to manual review, causing lost interest income, customer attrition, extra analyst workload, and possible fairness or regulatory scrutiny of declined applicants.
Giả định chi phí	No cost figures were provided, so we assume a missed bad debt is about [k] times costlier than a false alarm, which favours a lower threshold than 0.50 / a recall-oriented operating point.
Khi quy mô review có hạn	Because manual review capacity is limited, precision also matters: at threshold [t], about [x]% of flagged customers are truly bad debt (against a base rate of [y]%).

D3. Threshold

Lưu ý: ROC_AUC không phụ thuộc ngưỡng, còn F1 và Precision thì có. Vì vậy hàng Threshold là phần quan trọng của error analysis.

Cách chọn ngưỡng	Câu mẫu (English)
Giữ 0.50	We kept the default threshold of 0.50 because it was close to the validation optimum for F1 on label 1 ([t*]), so tuning offered no meaningful gain.
Tối ưu F1 label 1	We chose a threshold of [t] that maximised F1 for label 1 on [out-of-fold / validation] predictions, giving precision [p] and recall [r]. Raising it to [t2] would improve precision to [p2] but reduce recall to [r2].
Theo mục tiêu kinh doanh	A threshold of [t] was chosen to reach recall of at least [r] while keeping precision above [p], reflecting that missed bad debts are costlier than false alarms.
Cảnh báo lạc quan (nên có)	ROC_AUC is threshold-independent, whereas F1 and precision depend on the cutoff. Because the threshold was tuned on [validation / out-of-fold] data, F1 at this cutoff may be slightly optimistic.

D4. Caption hình (một câu)

- **Figure 1.** Validation confusion matrix at threshold [t] (left) and ROC curve with the chosen operating point (right); the dashed line marks a random classifier.
- **Figure 1.** Confusion matrix of [model] on the validation set at threshold [t]: [fn] bad-debt cases were missed and [fp] good customers were flagged; ROC_AUC = [x].

Chuẩn bị sẵn nội dung cho “1. Best Model Architecture and Workflow”

1.1 Data Preparation

Data and target

- **Dataset lớn (50k–100k rows), nhãn 1 ít (~5%):**
The dataset contains $[X]$ training rows and $[Y]$ features, with the binary target `bad_debt` defined as a customer whose debt payment is overdue for more than three months. The positive class (label 1) represents bad-debt customers and accounts for $[Z]\%$ of training samples, indicating a class-imbalanced problem.
- **Dataset vừa (10k–50k rows):**
The labelled training set comprises $[X]$ rows and $[Y]$ columns. The target `bad_debt = 1` denotes a customer with repayment overdue by more than three months, with a positive rate of $[Z]\%$.
- **Dataset nhỏ (<10k rows):**
Training data contains $[X]$ rows and $[Y]$ features. The target is binary (`bad_debt`): 1 if the customer's debt repayment is overdue by more than three months, 0 otherwise. The minority class represents $[Z]\%$ of rows.

Missing values

- **Tree-based model (XGBoost/RF/LightGBM):**
Missing values were left in place because gradient-boosted trees and random forests handle `NaN` natively through default split directions, avoiding distributional assumptions.
- **Linear/Logistic/SVM model:**
Numeric features were imputed with the median (robust to outliers), and categorical features with the mode or a dedicated "`Missing`" category to preserve the signal of missingness itself.
- **Mixed approach:**
For numeric features, missing values were imputed using the median within each fold to prevent leakage; for categorical features, an explicit "`Missing`" level was added so the absence of information could still be learned.

Encoding and scaling

- **One-hot encoding (low cardinality):**
Categorical features with fewer than $[N]$ unique values were one-hot encoded, producing $[K]$ binary columns. This avoids imposing an arbitrary ordinal relationship on nominal categories.
- **Target / frequency encoding (high cardinality):**
High-cardinality categorical features were target-encoded using the smoothed mean of the positive class, computed inside each CV fold to prevent leakage, with `smoothing = [N]`.
- **Scaling:**
Numeric features were standardised with `StandardScaler` (zero mean, unit variance) so that distance-based and regularised models were not biased by feature scale.

Features

- **Không tạo feature mới:**
No additional features were engineered; the final model relied on the original $[Y]$ columns after cleaning and encoding.
- **Có tạo feature:**
Two derived features were added: `debt_to_income` (current debt divided by declared income) and

`late_payment_ratio` (late payments over total payments), both computed without future information to avoid leakage. Features with variance below 0.001 or correlation above 0.95 with another feature were dropped.

- **Có loại feature:**

Identifiers and near-constant columns (e.g., `[col_name]`, `[col_name]`) were removed before modelling as they carry no predictive signal.

Imbalance

- **`class_weight='balanced'`:**

Class imbalance was addressed by setting `class_weight='balanced'`, which reweights each sample inversely to its class frequency so that the minority class contributes proportionally more to the loss.

- **SMOTE:**

The minority class was oversampled with SMOTE (`k_neighbors = 5`) only on the training fold, while the validation fold remained untouched, ensuring the resampling did not leak into evaluation.

- **`scale_pos_weight` (XGBoost):**

For XGBoost, `scale_pos_weight = N_neg / N_pos` was used to penalise errors on the positive class without modifying the data distribution.

- **Không xử lý imbalance:**

No explicit resampling was applied; the model's ability to handle imbalance comes from `class_weight='balanced'` and the choice of threshold at the prediction step.

1.2 Final Pipeline and Validation

Pipeline

- **Pipeline đơn giản:**

The final pipeline is `Raw inputs → [imputation / encoding / scaling] → [Algorithm] → probability → class label at threshold 0.5`.

- **Pipeline có SMOTE:**

The final pipeline is `Raw inputs → median/mode imputation → one-hot encoding → StandardScaler → SMOTE (training fold only) → [Algorithm] → probability → class label`.

- **Ensemble (Voting/Stacking):**

The final model is a soft-voting ensemble of [Model A, Model B, Model C]. Each base model is trained on the same preprocessed features, and the final probability is the arithmetic mean of the predicted probabilities. The base models were chosen for their complementary error patterns — linear (Logistic Regression), bagged trees (Random Forest), and gradient boosting (XGBoost).

- **Neural network (nếu dùng):**

The network is a feed-forward MLP with three fully-connected layers ([128, 64, 32] units, ReLU activation), a sigmoid output for binary classification, and 30% dropout after each hidden layer. It is trained with Adam ($lr=1e-3$), batch size 256, and early stopping on validation loss with patience of 5 epochs.

Model settings

- **Logistic Regression:**

Algorithm: Logistic Regression. Key hyperparameters: `C = [N]` (regularisation strength), `penalty = l2`, `solver = lbfgs`, `class_weight = balanced`.

- **Random Forest:**

Algorithm: Random Forest Classifier. Key hyperparameters: `n_estimators = [N]`, `max_depth = [N]`, `min_samples_leaf = [N]`, `class_weight = balanced`.

- **XGBoost:**
Algorithm: XGBoost Classifier. Key hyperparameters: `n_estimators = [N]`, `max_depth = [N]`, `learning_rate = [N]`, `scale_pos_weight = [N]`.
- **LightGBM:**
Algorithm: LightGBM. Key hyperparameters: `n_estimators = [N]`, `num_leaves = [N]`, `learning_rate = [N]`, `min_child_samples = [N]`.

Validation

- **5-fold Stratified CV:**
Validation used 5-fold stratified cross-validation, preserving the positive-class ratio in every fold. The reported F1, Weighted F1, and AP scores are the mean across folds with standard deviation in parentheses.
- **Holdout + CV:**
Validation combined a held-out 20% stratified validation set with 5-fold CV on the remaining 80%; the held-out scores were used as the final selection criterion to avoid overfitting model choice to CV noise.
- **Ngăn leakage:**
All preprocessing steps (imputation, encoding, scaling, and any resampling) were fitted only on the training fold and then applied to the validation fold through a `Pipeline` object, so no statistics from the validation data leaked into the model.

Selection and refit

- **Chọn theo Weighted F1:**
[Algorithm] was selected because it achieved the highest validation Weighted F1 ($[N] \pm [N]$) and AP ($[N] \pm [N]$) among the experiments actually completed, with stable per-fold variance.
- **Chọn theo trade-off:**
Although [Algorithm A] produced a marginally higher Weighted F1 (+0.01), [Algorithm B] was preferred for its much lower training time ($[N]$ s vs $[N]$ s) and simpler deployment, an acceptable trade-off given the small performance gap.
- **Refit:**
After selection, the chosen pipeline was refit on the entire labelled training set with the same hyperparameters before being used to generate the test predictions, maximising the data available to the final model.
- **Không refit:**
No final refit was performed; predictions on the test set were produced by averaging the predictions of the $[N]$ fold models, which is equivalent to a bagged estimator and avoids losing the early-stopping safeguard from the validation folds.

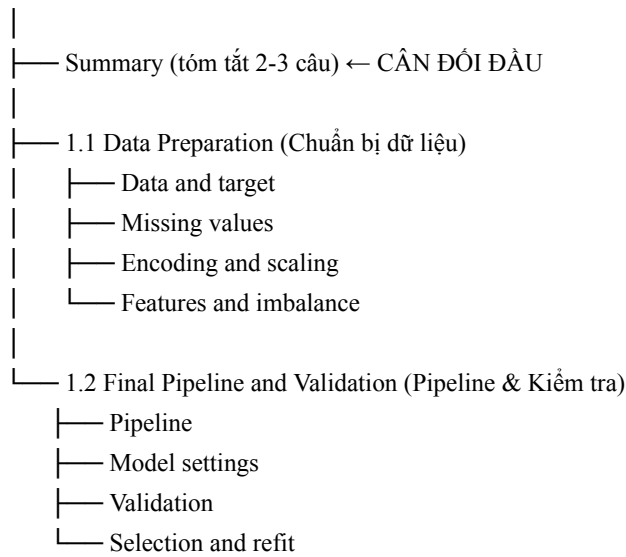
Hướng Dẫn Điền Template - Phiên Bản Cho Người Mới

1. Đọc hiểu template

Cấu trúc tổng quan

Template Part 1 có 2 phần chính, mỗi phần gồm 4 mục nhỏ:

1. Best Model Architecture and Workflow



2. Giải thích từng mục (dễ hiểu)

Summary (Tóm tắt)

Câu hỏi: "Tóm tắt ngắn gọn: Các bạn làm gì, tại sao chọn?"

Cách điền: Viết **2-3 câu**, mỗi câu trả lời:

1. Mô hình gì?
2. Kết quả validation ra sao?
3. Tại sao chọn mô hình này?

Ví dụ:

"We trained a Random Forest classifier with balanced class weights on the labelled training set. The model achieved a validation Weighted F1 of 0.84 and AP of 0.79 using 5-fold stratified cross-validation. It was selected over Logistic Regression and XGBoost because it provided the best trade-off between performance and training time."

Lỗi thường gặp:

-  Viết 1 đoạn dài 5-6 câu → Quá giới hạn 1 trang
-  Viết lý thuyết về classification, không nói gì về model cụ thể
-  Viết đúng 2-3 câu, mỗi câu trả 1 câu hỏi

1.1 Data Preparation

Data and target

Câu hỏi: "Dataset của các bạn như thế nào? Target nghĩa là gì?"

Cách đọc placeholder:

[Training rows / columns; target meaning and positive-label definition]
↓ ↓ ↓
[Số dòng] [Số cột] "Bad debt = nợ quá hạn 3 tháng"

Ví dụ điền:

*"The dataset contains 12,500 training rows and 23 features. The target **bad_debt** is binary: label 1 indicates a customer with debt payment overdue for more than three months; label 0 indicates no bad debt. The positive class (label 1) represents 8.3% of the training data."*

Missing values

Câu hỏi: "Chỗ trống trong dữ liệu xử lý thế nào?"

Hai trường hợp chính:

Model bạn dùng	Cách xử lý	Viết vào template
Tree-based (RF, XGBoost, LightGBM)	Để nguyên NaN	"Missing values were left in place..."
Linear (Logistic, SVM)	Điền số/chữ	"Numeric features were imputed with median..."

Ví dụ (tree-based):

*"Missing values were left in place because gradient-boosted trees handle **NaN** natively through default split directions, avoiding distributional assumptions."*

Ví dụ (linear):

"Numeric features were imputed with the median (robust to outliers), and categorical features with the mode or a dedicated 'Missing' category."

Encoding and scaling

Câu hỏi: "Dữ liệu chữ/đặc biệt xử lý sao? Có cần chuẩn hóa không?"

Ba bước suy nghĩ:

Bước 1: Có dữ liệu dạng chữ (categorical) không?

→ Có: Encode bằng gì?

→ Không: Bỏ qua phần encoding

Bước 2: Model có nhạy cảm với scale không?

→ Tree-based: Không cần scale

→ Linear/SVM/Neural: Cần StandardScaler

Bước 3: Encode bằng gì?

→ Low cardinality (ít loại): One-hot

→ High cardinality (nhiều loại): Target/Frequency encoding

Ví dụ (tree-based, có categorical):

"Categorical features with fewer than 10 unique values were one-hot encoded. Numeric features were not scaled as tree-based models are invariant to monotone transformations."

Ví dụ (linear, có categorical):

"Categorical features were one-hot encoded; numeric features were standardised with StandardScaler so that regularised models were not biased by feature scale."

Features and imbalance

Câu hỏi: "Có tạo feature mới không? Class mất cân bằng xử lý sao?"

Phần 1 - Feature Engineering:

Có tạo feature mới?

→ Có: "[Tên feature] = [công thức], [tính được bằng cách nào]"

→ Không: "No additional features were engineered"

Phần 2 - Class Imbalance:

Imbalance ratio = (Số class 0) : (Số class 1)

Imbalance nặng (> 10:1)? → Cần xử lý

Imbalance nhẹ (< 5:1)? → Có thể bỏ qua

Cách xử lý

Viết gì

`class_weight='balanced'`

"Class imbalance was addressed by setting
`class_weight='balanced'` ..."

SMOTE	"The minority class was oversampled with SMOTE (k_neighbors=5) on the training fold only..."
scale_pos_weight (XGBoost)	"For XGBoost, scale_pos_weight=[ratio] was used..."

1.2 Final Pipeline and Validation

Pipeline

Câu hỏi: "Dữ liệu đi qua những bước nào trước khi ra kết quả?"

Cách viết: Dùng mũi tên → để nối các bước

Ví dụ đơn giản (tree-based):

"Raw inputs → median/mode imputation → one-hot encoding → Random Forest → probability → class label at threshold 0.5"

Ví dụ phức tạp (ensemble):

"Raw inputs → imputation → one-hot encoding → StandardScaler → soft-voting ensemble (Logistic Regression, Random Forest, XGBoost) → averaged probability → class label"

Mẹo: Viết theo thứ tự thời gian, bắt đầu từ dữ liệu thô, kết thúc bằng kết quả.

Model settings

Câu hỏi: "Mô hình có những "nút điều chỉnh" quan trọng nào?"

Cách đọc: Mỗi tham số trong ngoặc là 1 hyperparameter

Ví dụ:

*"Algorithm: XGBoost Classifier. Key hyperparameters: **n_estimators=200, max_depth=6, learning_rate=0.1, scale_pos_weight=11.2.**"*

Các hyperparameters quan trọng theo model:

Model	Hyperparameters quan trọng
Logistic Regression	C (regularisation), solver
Random Forest	n_estimators, max_depth, min_samples_leaf

XGBoost	n_estimators, max_depth, learning_rate, scale_pos_weight
LightGBM	n_estimators, num_leaves, learning_rate
Neural Network	layers, units, learning_rate, dropout

Validation

Câu hỏi: "Các bạn kiểm tra model bằng cách nào? Làm sao tránh leakage?"

Hai phần cần viết:

Phần 1 - Phương pháp validation:

CV nhỏ (< 10k rows): 5-fold hoặc 10-fold Stratified CV

CV lớn (> 50k rows): 3-fold CV hoặc 80/20 split

Phần 2 - Cách tránh leakage:

Điều quan trọng: Mọi bước xử lý phải "fit trên train, transform trên validation"

Ví dụ (5-fold CV, tree-based):

"Validation used 5-fold stratified cross-validation. All preprocessing steps were fitted only on the training fold and applied to the validation fold through a Pipeline object, ensuring no statistics from validation data leaked into the model."

Ví dụ (holdout, linear model):

"Validation used a stratified 80/20 train/validation split. Imputation (median/mode), encoding, and StandardScaler were fit on the training set only and then applied to the validation set to prevent leakage."

Selection and refit

Câu hỏi: "Tại sao chọn model này? Có train lại trên toàn bộ data không?"

Hai phần:

Phần 1 - Selection rationale (Tại sao chọn?):

Chọn model nào?

→ Điểm cao nhất: "selected because it achieved the highest validation Weighted F1..."

→ Trade-off: "selected despite slightly lower F1 because of faster training time..."

→ Đơn giản hơn: "preferred for its simpler pipeline and lower risk of overfitting..."

Phần 2 - Refit (Có train lại không?):

Refit = Train lại model trên TOÀN BỘ training data

→ Có refit: "The chosen pipeline was refit on the entire labelled training set..."

→ Không refit: "No final refit was performed; predictions averaged across folds..."

3. Checklist trước khi nộp

- ☐ Summary: 2-3 câu, đề cập model + kết quả + lý do chọn
 - ☐ Data and target: Điền số rows/columns, giải thích target
 - ☐ Missing values: Chọn đúng cách theo model dùng
 - ☐ Encoding and scaling: Chọn đúng cách theo model dùng
 - ☐ Features and imbalance: Nói về feature mới (nếu có) + imbalance
 - ☐ Pipeline: Viết theo thứ tự từng bước
 - ☐ Model settings: Liệt kê hyperparameters quan trọng
 - ☐ Validation: Nói rõ phương pháp + cách tránh leakage
 - ☐ Selection and refit: Giải thích lý do chọn + có refit không
 - ☐ Độ dài: Không quá 1 trang cho toàn Part 1
-

4. Ví dụ hoàn chỉnh (điền đầy đủ)

1. Best Model Architecture and Workflow

We trained a Random Forest classifier with balanced class weights on the labelled training set. The model achieved a validation Weighted F1 of 0.84 and AP of 0.79 using 5-fold stratified cross-validation. It was selected over Logistic Regression and XGBoost because it provided the best trade-off between performance and training time.

1.1 Data Preparation

Data and target The dataset contains 12,500 training rows and 23 features. The target `bad\_debt` is binary: label 1 indicates a customer with debt overdue by more than three months; label 0 indicates no bad debt. The positive class represents 8.3% of training data.

Missing values Missing values were left in place because gradient-boosted trees handle NaN natively through default split directions.

Encoding and scaling Categorical features with fewer than 10 unique values were one-hot encoded. Numeric features were not scaled as tree-based models are scale-invariant.

Features and imbalance No additional features were engineered. Class imbalance (8.3% positive rate) was addressed using `'class_weight='balanced'` in Random Forest.

1.2 Final Pipeline and Validation

Pipeline Raw inputs → one-hot encoding → Random Forest Classifier → probability → class label at threshold 0.5.

Model settings Algorithm: Random Forest Classifier. Key hyperparameters: `'n_estimators=200'`, `'max_depth=12'`, `'min_samples_leaf=10'`, `'class_weight='balanced'`.

Validation 5-fold stratified cross-validation. Reported scores are means across folds with standard deviation in parentheses.

Selection and refit Random Forest was selected because it achieved the highest validation Weighted F1 (0.84 ± 0.02) among completed experiments. After selection, the pipeline was refit on the entire labelled training set before generating test predictions.

Bạn muốn mình xuất file này thành Markdown để bạn đọc/tham khảo khi thi không?